# Advanced Stock Trading and Portfolio Management Application

HCLTech Career Shaper | Project 25 | Individual academic project
Validated dataset snapshot: 2021-10-26 to 2026-10-26 | Report generated: 2026-10-26

## Abstract

Portfolio Lab is a working virtual trading and analysis application for eight NSE stocks. It integrates an auditable SQLite trade ledger, a cleaning pipeline, cached market history, portfolio P&L, risk measures, technical signals, a next-open backtest and a time-ordered classification model. The interface is a five-page Streamlit application with CSV exports and a consistent simulation warning.

The supplied source has 500 synthetic rows. Cleaning repaired 126 high and 159 low bounds. The active downloaded cache contains 11,123 validated stock and benchmark rows. The historical model evaluation is reported alongside a majority-class baseline and is not represented as investment performance.

Simulation only. NOT investment advice. Virtual money, no real brokerage connection.

# 1. Problem statement and objectives

A learner or portfolio analyst needs a safe place to explore how price data, trade accounting, risk and strategy testing fit together. A raw OHLCV table cannot directly answer how much virtual cash is left after fees, how a sale changes realised profit, or whether a trading signal would have survived realistic order timing. The project solves these questions with reproducible calculations and a local database, without placing real orders.

## Measurable objectives

| Objective | Acceptance evidence |
|---|---|
| Clean the supplied data | 500 rows preserved; invalid candle bounds repaired |
| Maintain a virtual portfolio | Atomic orders, weighted average cost and cash/share guards |
| Quantify risk | Six historical measures and NIFTY 50 benchmark where available |
| Evaluate signals honestly | Next-open fills, fees and buy-and-hold comparison |
| Demonstrate ML discipline | Chronological holdout, label gap, baseline and confusion matrix |
| Deploy reproducibly | Pinned dependencies, Docker Compose and a health endpoint |

## Project boundaries

The application uses virtual money only. It has no broker integration, real-money execution, authentication or personal-data collection. Cached yfinance prices are historical observations rather than live quotes. The app displays a simulation disclaimer on every page. Academic results are descriptive and do not justify a recommendation to trade.

## How the five views answer the problem

Dashboard answers what the account is worth and why it changed. Trade provides an auditable cash-and-share ledger. Risk turns a price series into definitions that can be compared with the NIFTY 50 benchmark. Signals separates a visual indicator from its next-open execution rule. Data Explorer exposes the rows and quality decisions behind all other views. The same SQLite database underlies each page, so state is consistent across navigation and restarts.

## 2. Data provenance and quality

The original `dataset.csv` has eight ticker symbols and columns for date, OHLC prices, volume and a provided daily return. The dates cover 63 consecutive calendar days in early 2024, so weekend rows reveal its synthetic nature. The original files were preserved. The pipeline writes `cleaned_dataset.csv`, recalculates returns by ticker and records all changes in `quality_report.json`.

| Quality check | Observed / action |
|---|---|
| Rows / duplicate keys | 500 / 0 |
| Invalid high bounds | 126 repaired using max(high, open, close) |
| Invalid low bounds | 159 repaired using min(low, open, close) |
| Weekend rows | 140 retained and labelled synthetic |
| Invalid rows dropped | 0 |
| Active history | 11,123 rows from yfinance |

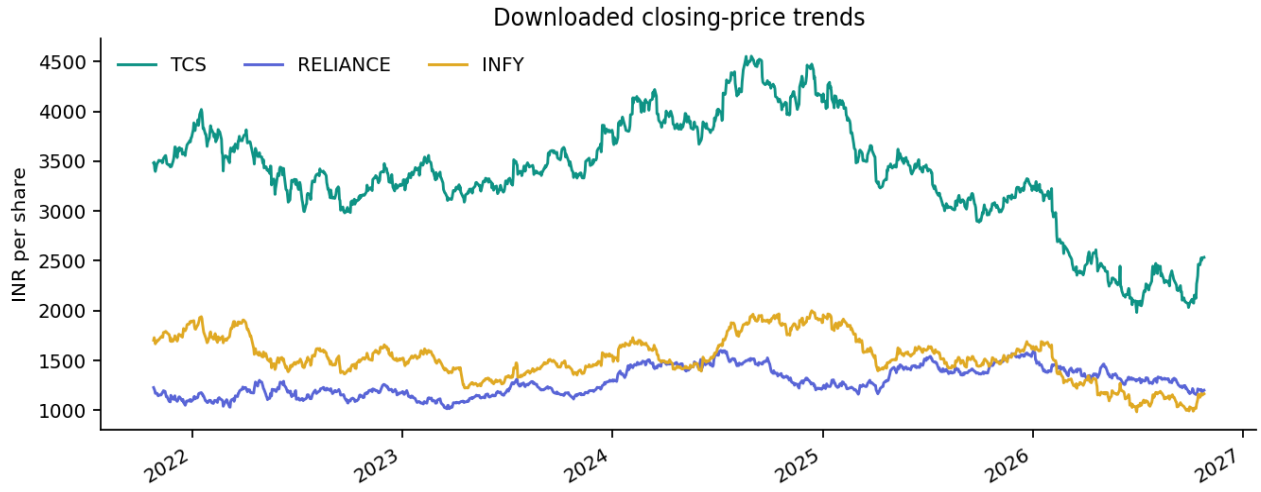
When available, yfinance supplies five years of `.NS` history for the eight stocks and `^NSEI` for the benchmark. Activation requires at least 252 rows for each stock and for NIFTY 50. The real cache is kept separate from the synthetic CSV. If download or completeness checks fail, the supplied data remains a functional fallback and annualised measures and ML are withheld.

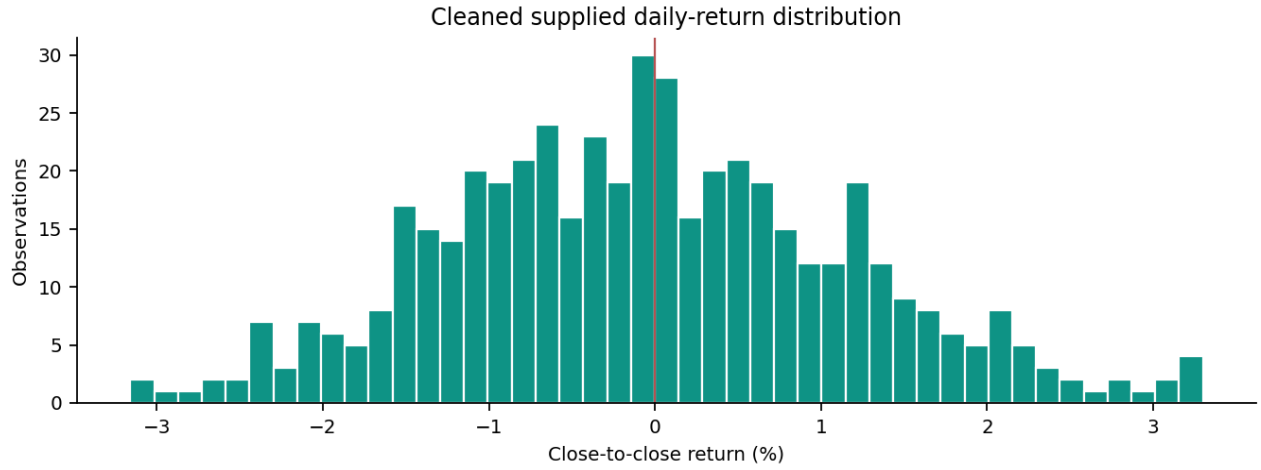
After cleaning, each ticker's first close-to-close return is undefined. Volume must be nonnegative and OHLC values positive. SQLite also checks candle order and uniqueness of `(date, ticker)`.

The cleaning function is vectorised with pandas. It parses dates, coerces numeric fields, drops invalid identifiers or nonpositive prices, deduplicates date-ticker keys, expands candle bounds to include open and close, and recomputes returns after sorting by ticker and date. The quality report counts these actions, making repairs observable rather than silently altering the source. No synthetic row is appended to a downloaded price series.

# 3. Exploratory analysis: prices and returns

Price levels differ substantially by company and can shift over time; direct comparisons of levels are therefore descriptive rather than risk adjusted. The first figure shows three stock close series from the active downloaded cache. The second displays the recomputed returns of the supplied synthetic data. Its short period and artificial calendar mean should not be interpreted as a market distribution.

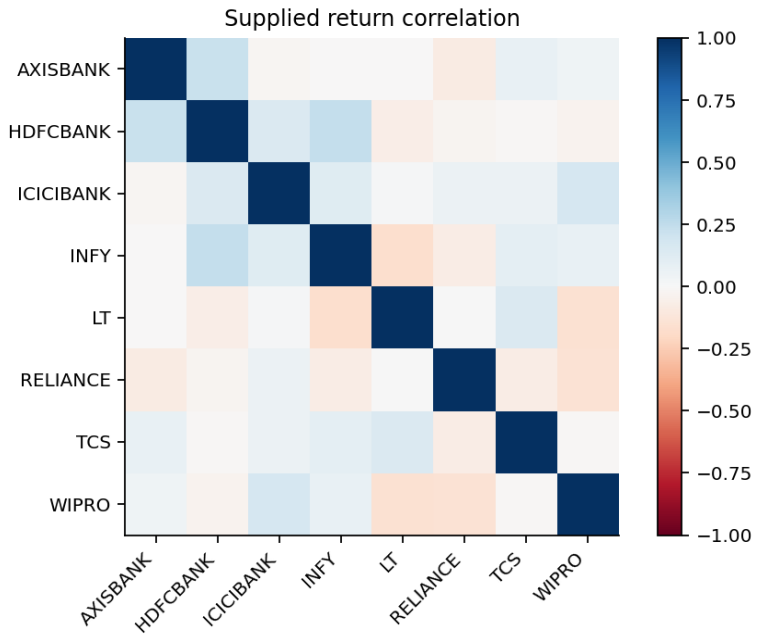


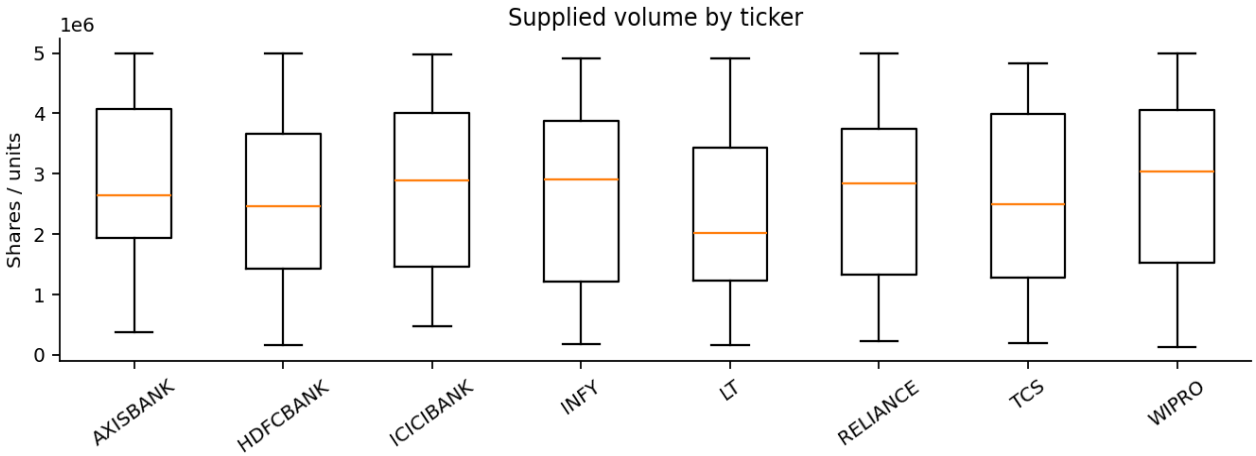




The executable notebook contains these views plus a correlation heatmap and a volume comparison. The source separation is deliberate: repaired supplied data supports the academic data-cleaning exercise, while the longer real cache supports annualised calculations.

# 4. Exploratory analysis: co-movement and volume

Pairwise correlations use observations shared by two ticker return series. They describe this sample only; they can change with the date window, missingness and market regime. Volume distributions show that the supplied dataset has different activity scales across tickers, which motivates per-ticker inspection instead of treating all volumes as directly comparable.



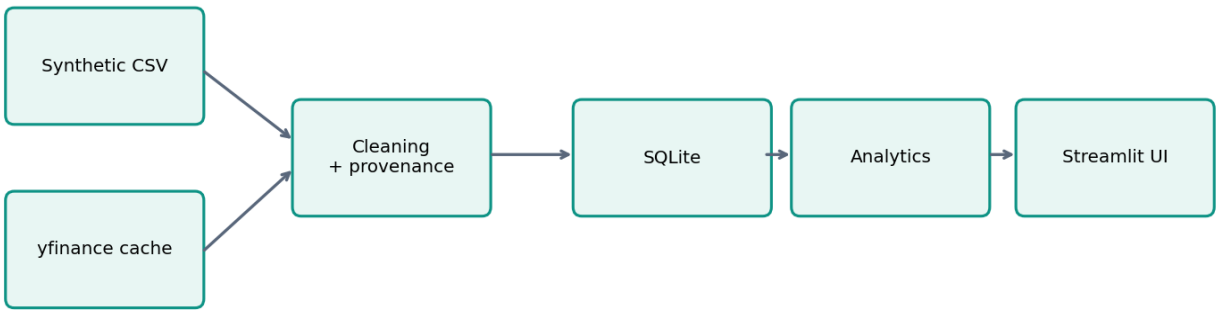
# 5. Architecture and persistence

Streamlit pages call independent Python modules. The data pipeline validates sources and writes SQLite prices. The portfolio module owns orders and valuation; risk and signals read the same prices; the ML module builds features from past and current observations. `docs/architecture.md` includes the level-0 data-flow diagram and the block diagram in Mermaid form.

| Layer | Responsibility |
|---|---|
| Streamlit UI | Dashboard, Trade, Risk, Signals, Data Explorer; shared theme and warning |
| Pipeline | CSV cleaning, yfinance cache, provenance and quality report |
| SQLite | Stocks, prices, portfolio, transactions, signals and holdings view |
| Analytics | P&L;, risk, technical indicators, backtest and classifier |
| Deployment | Python 3.11 slim image, non-root user, bind-mounted data, health check |

## Transaction integrity

Orders begin an immediate SQLite transaction before reading cash or shares. A buy checks that its notional plus fee fits cash; a sell checks owned quantity. Order insertion and cash adjustment then commit together. Foreign keys, primary keys, indexes and CHECK constraints reject inconsistent records. SQL parameters are bound rather than interpolated.




## Reproducibility

A clean clone can be started with `docker compose up --build`. The first container start initializes the database and signals, then launches Streamlit. The data directory is mounted so trades and cached history survive container replacement. If the network is unavailable, the supplied data still allows a working demonstration.

# 6. Portfolio accounting and risk

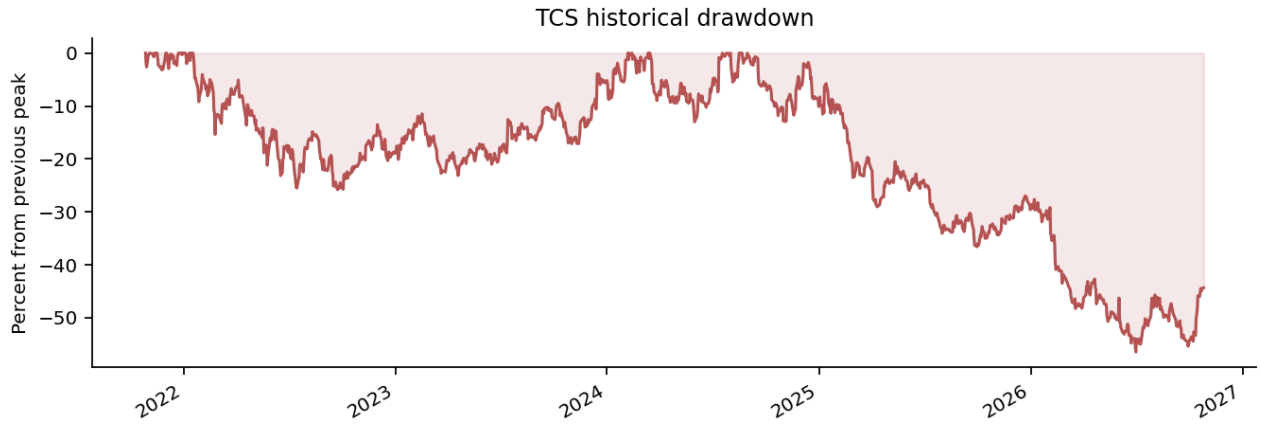
The demo portfolio begins with INR 1,000,000 cash. Each buy adds notional and brokerage to its remaining cost basis. Each sell removes quantity at the moving average cost; realised P&L is net proceeds minus the removed cost. Unrealised P&L is current market value minus remaining basis. Total P&L is current cash plus holdings value minus initial funding. A holding's weight is its market value divided by total portfolio value.

The test suite verifies an example: buy 10 shares at INR 100, sell 4 at INR 120, and charge INR 20 on each order. The six remaining shares have INR 102 average cost; realised P&L is INR 52, unrealised P&L at INR 120 is INR 108 and total P&L is INR 160.

## Historical risk formulas

Annualised volatility = sample standard deviation of daily returns x sqrt(252). Sharpe = (mean daily return - 0.065/252) / daily standard deviation x sqrt(252). Beta = covariance(asset, NIFTY 50) / variance(NIFTY 50) on overlapping dates. Maximum drawdown = minimum of compounded wealth / its previous peak - 1. Historical 95% VaR = max(0, minus the fifth percentile daily return). Correlation uses Pearson's coefficient.

| TCS risk measure (1235 returns) | Value |
|---|---|
| Annualised volatility | 22.87% |
| Sharpe, 6.5% risk-free | -0.453 |
| Beta vs NIFTY 50 | 0.821 |
| Maximum drawdown | -56.46% |
| Historical one-day VaR 95% | 2.10% |




Current shares and cash are applied to historical closes for the portfolio risk view. That is an exposure scenario, not the realised account value path. All risk figures are historical sample estimates.

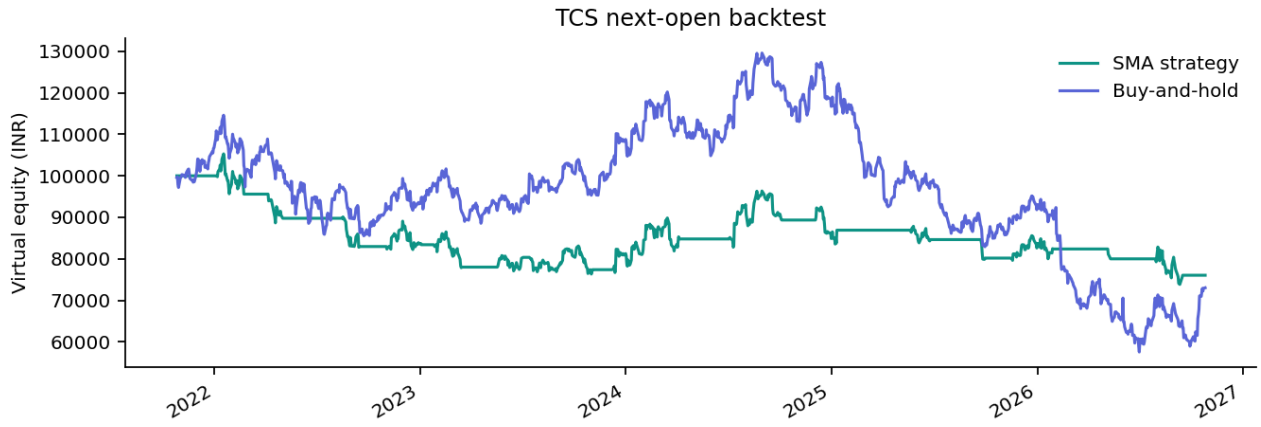
# 7. Signal simulation and machine learning

The SMA strategy emits buy and sell signals when the 20-day mean crosses the 50-day mean. RSI(14) uses Wilder-style smoothing and marks oversold observations below 30 or overbought observations above 70. A signal is observed only after the day's close; the backtest trades at the next open. It remains long or in cash, buys whole shares and charges the portfolio's brokerage model. Buy-and-hold purchases at the first open for comparison.

| TCS SMA backtest measure | Observed result |
|---|---|
| Strategy total return | -23.94% |
| Buy-and-hold return | -26.96% |
| Completed round trips | 15 |
| Win rate | 33.33% |
| Maximum drawdown | -29.86% |

The next-day classifier uses close-known price, volume, momentum, range, RSI and rolling-volatility features. A scaler and logistic regression are fitted on earlier rows only. The last training label is left before the test period, preventing it from using a test-period close. The final 20% is evaluated once, without random shuffling.

| TCS holdout measure | Observed result |
|---|---|
| Train / test rows | 946 / 237 |
| Accuracy / majority baseline | 52.74% / 53.59% |
| Precision / recall / F1 | 0.493 / 0.673 / 0.569 |




These values are a single historical sample. The classifier is not deployed as an order generator, and a score below the majority baseline provides no evidence of a trading advantage.

# 8. Verification, limitations and conclusion

Focused pytest checks cover cleaning, SQLite constraints, brokerage-aware cost basis, order guards, known beta and drawdown values, next-open execution and chronological ML evaluation. Streamlit's test runner executed the home page and all five feature pages without app exceptions. A live Streamlit server returned HTTP 200 from `/_stcore/health`. Docker Compose configuration passed validation. Container image execution still requires an active Docker daemon on the host.

The test data includes a fully known two-day order scenario and an artificial overnight price jump that would expose same-day signal execution. These checks are more informative than tests that merely restate the implementation. The app pages were also exercised against the longer cached history to confirm that the risk cards, chart generation and ML evaluation can render with actual observations.

## Limitations

Historical prices are cached and may be revised by the provider. The source CSV is synthetic and too short for annualised inference. The backtest uses a single selected ticker, has no slippage, taxes, partial fills, liquidity constraints or corporate-action treatment beyond the provider's downloaded values. A one-off chronological ML holdout has no uncertainty interval or walk-forward validation. Risk estimates are sensitive to sample period and do not limit future loss.

## Ethical compliance

The system uses only public price data and a virtual account. It collects no names, credentials or other personal data, has no live broker connection and displays the requested disclaimer on every page. The learner must personally review the final submission, disclose tool assistance where required and sign the separate ethical declaration; this generated report does not claim a personal signature.

## References

[1] HCLTech Career Shaper, Project Brief and Dataset Generation Prompt, supplied in `data/`.

[2] Streamlit, Creating multipage apps using the pages directory.

[3] pandas, DataFrame.pct_change documentation.

[4] scikit-learn, TimeSeriesSplit documentation.

Simulation only. NOT investment advice. Virtual money, no real brokerage connection.